



SENIOR SOFTWARE ENGINEER INTERVIEW QUESTIONS: DEEP DIVES

Transactions and the Outbox

The senior engineer answer sheet

The row committed. The event never published. Two systems, one try block, no atomicity.

Before saying Kafka transaction, outbox, or distributed transaction, draw the state boundary. Which systems must agree, and which may be allowed to catch up?

Most arguments about outbox versus transactions are really arguments about where somebody drew that boundary without saying so.

This sheet is the written form of the episode. Kafka behaviour is sourced to Apache Kafka 4.3, database behaviour to PostgreSQL 17, and reasoning that is reasoning says so.

In scope

- The dual write, and why a try block is not a transaction
- What a Kafka transaction genuinely covers
- `read_committed`, the half of the guarantee on the consumer
- Where the atomicity boundary stops
- The transactional outbox, and what it actually buys
- Why the outbox is not exactly-once publication
- Polling against change data capture

Out of scope

- `transactional.id` fencing specifics, which were not verified for this sheet
- Named CDC products, versions and configuration
- Saga and compensating transactions, which are a different answer
- Kafka Streams exactly-once, which manages this for you

A sheet that quietly asserts unverified fencing behaviour would be wrong in someone hands for years. Absence here is a choice.



01 Worked assumptions

Payments per day	50,000
Dual write failure rate assumed	1 in 10,000
Silent inconsistencies per day at that rate	5
Inconsistencies per year	about 1,800
Outbox poll interval	200 ms
Publication latency added by polling	up to one interval

These are interview assumptions chosen to make the behaviour concrete. They are not published figures from any provider and no claim is made about any particular deployment.

03 The dual write, and why a try block is not a transaction

Save the payment to the database. Publish the event to Kafka. Both inside one try block, and it looks atomic.

It is not. Two transaction managers in one try block is two transactions. The database commits, the publish throws, and the catch block cannot undo a commit that already happened.

Reverse the order and you get the other failure: the event is out, consumers act on it, and the row never lands. Now downstream believes in a payment that does not exist.

Two transaction managers coordinated inside one try block is not one transaction, because neither participant can roll back on behalf of the other.

First principles, not vendor documentation

A transaction bundles multiple steps into a single all-or-nothing operation only for the resources enrolled in that transaction.

PostgreSQL 17, Transactions

04 What a Kafka transaction genuinely covers

A Kafka transaction is real and it is precise. A transactional producer can write to several topics and partitions and commit the consumed input offsets, all atomically, in one transaction.

That is exactly the consume, process, produce loop, and inside Kafka it works.

What it does not do is reach your database or an HTTP call. Kafka own documentation is explicit about that boundary, which is why the honest interview reply to "we enabled transactions" is a question: exactly once ACROSS WHAT?

A Kafka transaction can atomically produce records to multiple topic partitions and commit the input offsets for the consumed records in the same transaction.

Kafka 4.3, Design: Transactions

Exactly-once delivery for other destination systems generally requires cooperation with such systems, but Kafka provides the primitives which makes implementing this feasible.

Kafka 4.3, Design: Message Delivery Semantics

05 The half of the guarantee that lives on the consumer

A transactional producer alone guarantees nothing a reader can observe. The consumer has to be on `read_committed` for aborted records to be hidden from it.

On the default, `read_uncommitted`, a consumer sees records from transactions that were later aborted. The producer did everything right and the reader acted on rolled back work.

So the guarantee is a pair. Naming only the producer half is the most common way this topic is answered incorrectly while sounding correct.

`isolation.level` controls how to read messages written transactionally, and `read_committed` is required for a consumer to see only committed transactional records.

Kafka 4.3, Consumer Configs

`read_committed` will only return messages up to the last stable offset, which is one less than the offset of the first open transaction.

Kafka 4.3, Consumer Configs

06 The distributed transaction answer, and why it is rarely the answer

Two phase commit exists. XA is a real specification and it does solve the dual write, provided every participant supports it.

That proviso is the problem. It requires participant support, it holds locks across the prepare phase, and a coordinator failure between prepare and commit leaves participants blocked.

Mentioning XA and then explaining why you would not reach for it here is a stronger answer than not knowing it exists.

Two phase commit requires every participating resource manager to implement the protocol, and the transaction manager to coordinate a prepare phase before commit.

The XA specification, Distributed Transaction Processing

A coordinator failure between prepare and commit leaves participants holding locks in a prepared state until the outcome is resolved.

The XA specification, Distributed Transaction Processing



07 The transactional outbox

Write the business row and the event row in ONE database transaction, to the same database. Both or neither. There is no second system to fail.

Then a separate relay reads unpublished rows, publishes them to Kafka, and marks them published.

That is the whole pattern, and its strength is that the atomic part now involves exactly one transactional resource.

The outbox writes the business row and the event row in one database transaction, so a failure in either leaves neither applied.

PostgreSQL 17, Transactions

The outbox pattern removes the dual write by ensuring the only atomic requirement is within a single transactional resource.

First principles, not vendor documentation

08 What the outbox does not buy, and most explanations skip this

The outbox does not give you exactly-once PUBLICATION.

Publish then mark is two steps. The relay can publish and die before marking, and the next pass publishes again. That is at-least-once publication, by construction.

Which is fine, and it is fine for one reason: the consumer is idempotent. The outbox guarantees the event WILL be published, not that it will be published once. Episode 5 is what makes that acceptable.

Publishing and then marking as published is two steps, so a relay failure between them results in the event being published more than once.

First principles, not vendor documentation

At-least-once publication is safe only where the consumer effect is idempotent; the outbox depends on that property rather than providing it.

First principles, not vendor documentation

09 Polling against change data capture

Polling the outbox table is a query on a loop. It adds latency up to one poll interval and load proportional to how often you ask.

Reading the database log instead removes the polling and adds infrastructure that has to be operated.

This is an operational judgement rather than documented behaviour, and it is flagged as one: most teams should start with polling, because it is debuggable at three in the morning by anyone who can read SQL.

Polling adds publication latency bounded by the poll interval and query load proportional to the polling frequency.

First principles, not vendor documentation

Choosing polling first for operability rather than for throughput is an operational judgement, not a documented recommendation.

Operational practice, not vendor documentation

10 The interview answer

Start by drawing the boundary: which systems must agree atomically, and which may catch up.

A Kafka transaction covers records and input offsets inside Kafka, and needs `read_committed` on the consumer to mean anything. It does not reach your database.

For a database plus Kafka, use the outbox: one transaction, one resource, then a relay.

Then say the honest part: the outbox publishes at least once, and that is only acceptable because the consumer is idempotent.

Volunteering that the outbox is at-least-once publication is the single strongest signal available on this topic, because the pattern is usually presented as if it were exactly once.

First principles, not vendor documentation

Questions to ask before you answer

1. Which systems must agree atomically, and which may lag?
2. Is there more than one transaction manager in this code path?
3. Are the consumers on read_committed?
4. Where exactly does the atomicity boundary stop?
5. Is the consumer idempotent, and keyed on what?
6. Who publishes the outbox, and what happens when it dies?
7. What is the age of the oldest unpublished row, and who watches it?
8. Is anything holding a Kafka transaction open long enough to block readers?

Everything else



Site

<https://code-with-sam-dev.github.io>



YouTube

<https://www.youtube.com/@CodewithSam-Dev>



LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>



X

<https://x.com/CodeWithSamDev>



TikTok

<https://www.tiktok.com/@codewithsamdev>



Instagram

<https://www.instagram.com/codewithsamdev>



Facebook

<https://www.facebook.com/codewithsamdev>



GitHub

<https://github.com/code-with-sam-dev>



Buy me a coffee

<https://buymeacoffee.com/codewithsam>



Buy me sadza

<https://buymesadza.com/codewithsam>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which system you want taken apart next.

Apache Kafka and the Kafka logo are trademarks of the Apache Software Foundation. PostgreSQL and its logo are trademarks of the PostgreSQL Community Association. They appear here only to identify the technology being discussed. Nothing here implies endorsement by, or affiliation with, any of them.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.